

Solution Requirements

Date	29 June 2026
Team ID	N/A
Project Name	OptiCrop smart agricultural production optimization engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	Farmers can securely log in to OptiCrop using their username and password to access crop recommendation features.	High
2	Crop Recommendation	The system accepts N, P, K, temperature, humidity, pH, and rainfall as inputs and returns the most suitable crop recommendation using the trained Random Forest model.	High
3	Multi-Algorithm Comparison	The system trains and compares multiple ML algorithms (Logistic Regression, KNN, Decision Tree, Random Forest, K-Means Clustering) and selects the best-performing model for deployment.	High
4	Transaction Processing	Processes user-submitted soil and climate inputs in real time and generates a crop prediction result within 3 seconds using the deployed model.pkl, scaler.pkl, and label_encoder.pkl artifacts.	High
5	Reporting & Results Display	Displays the recommended crop name on the Find Your Crop page after form submission. Prediction results are shown clearly with no ML jargon for farmer usability.	Medium
6	Business Rules	All crop recommendations are generated exclusively by the trained AI model based on the entered soil and climate parameters — no manual overrides or static lookup tables are used.	High
7	Data Privacy & Compliance	User input data is not stored permanently and is only used for the duration of the prediction session. The application follows standard web security practices for form data handling.	Medium
8	Accessibility	Users can access the crop recommendation system at any time via the web browser at localhost without requiring any installation or ML expertise.	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Crop recommendation should be generated quickly after the user submits the input form with all seven soil and climate parameters.	Response time < 3 seconds
2	Model Accuracy	The deployed machine learning model must achieve a high prediction accuracy on held-out test data to ensure reliable and trustworthy crop recommendations for farmers.	Test accuracy $\geq 95\%$ (achieved: 99.55% with Random Forest)
3	Scalability	The Flask application and trained model must support multiple concurrent user requests without degrading prediction accuracy or response time.	Supports up to 500 concurrent users
4	Security & Data Privacy	User-submitted form data (soil and climate values) must not be stored permanently. Input validation must reject non-numeric or empty form submissions gracefully.	No persistent data storage; input validation on all 7 fields
5	Reliability & Availability	The application must run reliably on the local development server without crashes during normal usage. The trained model artifacts must load correctly on every application restart.	Zero model-loading failures; no unhandled crashes during normal operation
6	Usability & Accessibility	The web interface must be simple enough for a non-technical farmer to use without any guidance. Labels must be in plain language with clear input field descriptions.	All 7 input fields clearly labelled; prediction result displayed in plain English
7	Maintainability	The ML pipeline must be fully reproducible — swapping the model file (model.pkl) with a retrained version must not require any changes to the Flask application code.	Model can be updated by replacing .pkl files only — zero code changes required
8	Portability	The application must run on Windows, macOS, and Linux using the same requirements.txt without platform-specific configuration changes.	Cross-platform compatible via Python 3.x virtual environment